Text Generation Using a Transformer Languag Model

1. Imports and Environment Setup

In [3]:
```
import os
import re
import math
import random
import numpy as np
import pandas as pd
from collections import Counter, defaultdict
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)
```

2. Dataset Loading and Exploration

In [4]:
```
dataset_candidates = [
    'articles_dataset.csv',
    'Text_Generation/articles_dataset.csv',
    '../articles_dataset.csv',
    '../Text_Generation/articles_dataset.csv',
    '../data/dataset.csv',
    'data/dataset.csv'
]

dataset_path = None
for path in dataset_candidates:
    if os.path.exists(path):
        dataset_path = path
        break

if dataset_path is not None:
    df = pd.read_csv(dataset_path)
else:
    articles_data = [
        [1, "technology", "Artificial intelligence and machine learning m
        [2, "technology", "Deep learning neural networks require massive
        [3, "technology", "Natural language processing allows computers t
        [4, "technology", "Cloud computing providers offer scalable virtu
        [5, "technology", "Python is a popular programming language for d
        [6, "technology", "Autonomous systems rely on computer vision alg
        [7, "technology", "Transformer models utilize self attention mech
        [8, "technology", "Modern language models predict the next token
        [9, "technology", "Software engineers design modular applications
```

```
        [10, "technology", "Cybersecurity analysts deploy encryption prot
        [11, "space", "Astronomers discover new planets and distant galax
        [12, "space", "NASA and private aerospace companies launch rocket
        [13, "space", "The solar system consists of the Sun, eight planet
        [14, "space", "Astronauts aboard the international space station
        [15, "space", "Black holes possess immense gravitational pull suc
        [16, "space", "Space probes travel billions of kilometers across
        [17, "space", "Astrophysicists study cosmic microwave background
        [18, "space", "Satellite constellations orbit planet Earth to pro
        [19, "space", "Robotic rovers on Mars collect soil samples, analy
        [20, "space", "Deep space exploration requires advanced propulsio
        [21, "health", "Regular physical exercise, balanced nutrition, an
        [22, "health", "Medical researchers develop novel vaccines, antib
        [23, "health", "Doctors and healthcare professionals recommend a
        [24, "health", "Mental health awareness encourages mindfulness me
        [25, "health", "Hospitals utilize advanced medical diagnostics, m
        [26, "health", "Cardiologists emphasize aerobic exercise such as
        [27, "health", "Public health organizations monitor epidemiologic
        [28, "health", "Adequate hydration and proper recovery periods ar
        [29, "health", "Clinical trials evaluate the safety, dosage, and
        [30, "health", "Preventive medicine focuses on lifestyle interven
        [31, "sports", "Football players train rigorously on tactical for
        [32, "sports", "The Olympic games bring together global athletes
        [33, "sports", "Basketball teams focus on fast breaks, perimeter
        [34, "sports", "Tennis champions master powerful serves, baseline
        [35, "sports", "Marathon runners require intense cardiovascular e
        [36, "sports", "Professional sports franchises use performance an
        [37, "sports", "Strength training builds muscular power, improves
        [38, "sports", "Coaches analyze game footage to devise defensive
        [39, "sports", "Swimming competitors practice stroke mechanics, u
        [40, "sports", "Cycling races test endurance, aerodynamic efficie
        [41, "finance", "Stock market investors analyze corporate financi
        [42, "finance", "Central banks manage monetary policy, inflation
        [43, "finance", "Venture capital funds invest financial capital i
        [44, "finance", "Cryptocurrency assets, digital tokens, and distr
        [45, "finance", "Global trade, supply chain logistics, and foreig
        [46, "finance", "Portfolio managers diversify asset allocations a
        [47, "finance", "Algorithmic trading firms execute high frequency
        [48, "finance", "Financial analysts forecast economic trends usin
        [49, "finance", "Fintech startups offer mobile payment solutions,
        [50, "finance", "Risk management frameworks help commercial banks
    ]
    df = pd.DataFrame(articles_data, columns=["id", "category", "text"])
    df.to_csv("articles_dataset.csv", index=False)
    dataset_path = "articles_dataset.csv"

print(f"Loaded dataset from: {dataset_path}")
print(f"Dataset Shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
print()
print(df.info())
df.head()
```

```
Loaded dataset from: articles_dataset.csv
Dataset Shape: (50, 3)
Columns: ['id', 'category', 'text']

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 3 columns):
 #   Column    Non-Null Count  Dtype 
---  ------    --------------  ----- 
 0   id        50 non-null     int64 
 1   category  50 non-null     object
 2   text      50 non-null     object
dtypes: int64(1), object(2)
memory usage: 1.3+ KB
None
```

Out[4]:

	id	category	text
0	1	technology	Artificial intelligence and machine learning m...
1	2	technology	Deep learning neural networks require massive ...
2	3	technology	Natural language processing allows computers t...
3	4	technology	Cloud computing providers offer scalable virtu...
4	5	technology	Python is a popular programming language for d...

3. Text Column Identification and Extraction

In [5]:
```
text_column_name = 'text'
raw_documents = df[text_column_name].dropna().astype(str).tolist()

print(f"Extracted text column: '{text_column_name}'")
print(f"Total documents extracted: {len(raw_documents)}")
print()
for i in range(min(5, len(raw_documents))):
    print(f"Document {i + 1}: {raw_documents[i]}")
```

```
Extracted text column: 'text'
Total documents extracted: 50

Document 1: Artificial intelligence and machine learning models are revolutio
modern computer science and software development.
Document 2: Deep learning neural networks require massive computing power and
graphics processing units for training complex architectures.
Document 3: Natural language processing allows computers to understand, analy
and generate human languages effectively across diverse domains.
Document 4: Cloud computing providers offer scalable virtual servers, storage
databases, and artificial intelligence application programming interfaces.
Document 5: Python is a popular programming language for data science, artifi
intelligence, automation, and web development frameworks.
```

4. Text Preprocessing and Vocabulary Construction

In [6]:
```
def clean_and_tokenize(text):
    normalized = text.lower()
    normalized = re.sub(r'[^a-z0-9\s]', ' ', normalized)
    tokens = [t for t in normalized.split() if len(t) > 0]
```

```
    return tokens

tokenized_documents = [clean_and_tokenize(doc) for doc in raw_documents]
all_tokens = [tok for doc in tokenized_documents for tok in doc]

special_tokens = ['<pad>', '<unk>']
unique_words = sorted(list(set(all_tokens)))
vocab = special_tokens + unique_words

word2idx = {word: idx for idx, word in enumerate(vocab)}
idx2word = {idx: word for idx, word in enumerate(vocab)}

print(f"Total token occurrences in corpus: {len(all_tokens)}")
print(f"Vocabulary size (including special tokens): {len(vocab)}")
print(f"Sample vocabulary entries: {vocab[:15]}")
```

```
Total token occurrences in corpus: 757
Vocabulary size (including special tokens): 530
Sample vocabulary entries: ['<pad>', '<unk>', 'a', 'ability', 'aboard', 'accu
'across', 'adequate', 'advance', 'advanced', 'aerobic', 'aerodynamic', 'aeros
'affect', 'aggressive']
```

5. Experimentation with Different Sequence Lengths

In [7]:

```
def extract_sequence_pairs(token_list, seq_len):
    sequences = []
    next_words = []
    for i in range(len(token_list) - seq_len):
        sequences.append(token_list[i : i + seq_len])
        next_words.append(token_list[i + seq_len])
    return sequences, next_words

candidate_lengths = [3, 5, 8]
experiment_summary = []

for length in candidate_lengths:
    seqs, targets = extract_sequence_pairs(all_tokens, length)
    experiment_summary.append({
        'Sequence Length': length,
        'Total Training Samples': len(seqs),
        'Sample Input Sequence': ' '.join(seqs[0]),
        'Sample Next Word': targets[0]
    })

exp_df = pd.DataFrame(experiment_summary)
exp_df
```

Out[7]:

	Sequence Length	Total Training Samples	Sample Input Sequence	Sample
0	3	754	artificial intelligence and	ma
1	5	752	artificial intelligence and machine learning	m
2	8	749	artificial intelligence and machine learning m...	m

6. Storing Sequences and Possible Next Words

In [8]:
```
chosen_seq_len = 5
sequence_target_map = defaultdict(list)

train_sequences, train_next_words = extract_sequence_pairs(all_tokens, ch

for seq, target in zip(train_sequences, train_next_words):
    key = tuple(seq)
    sequence_target_map[key].append(target)

print(f"Selected Sequence Length: {chosen_seq_len}")
print(f"Total unique sequence keys stored: {len(sequence_target_map)}")
print(f"Total sequence-to-target pairs: {len(train_sequences)}")
print()

for i, (seq_tuple, target_list) in enumerate(list(sequence_target_map.ite
    seq_representation = " ".join(seq_tuple)
    print(f"Sequence {i + 1}: '{seq_representation}' -> Next Word Candida
```

```
Selected Sequence Length: 5
Total unique sequence keys stored: 752
Total sequence-to-target pairs: 752

Sequence 1: 'artificial intelligence and machine learning' -> Next Word Candi
['models']
Sequence 2: 'intelligence and machine learning models' -> Next Word Candidate
['are']
Sequence 3: 'and machine learning models are' -> Next Word Candidates:
['revolutionizing']
Sequence 4: 'machine learning models are revolutionizing' -> Next Word Candid
['modern']
Sequence 5: 'learning models are revolutionizing modern' -> Next Word Candida
['computer']
Sequence 6: 'models are revolutionizing modern computer' -> Next Word Candida
['science']
```

7. PyTorch Dataset and DataLoader

In [9]:
```
class LanguageModelingDataset(Dataset):
    def __init__(self, sequences, targets, word_to_id):
        self.x_tensors = []
        self.y_tensors = []
        unk_id = word_to_id['<unk>']
        for seq, target in zip(sequences, targets):
            x_ids = [word_to_id.get(w, unk_id) for w in seq]
            y_id = word_to_id.get(target, unk_id)
            self.x_tensors.append(torch.tensor(x_ids, dtype=torch.long))
            self.y_tensors.append(torch.tensor(y_id, dtype=torch.long))

    def __len__(self):
        return len(self.x_tensors)

    def __getitem__(self, idx):
        return self.x_tensors[idx], self.y_tensors[idx]

dataset = LanguageModelingDataset(train_sequences, train_next_words, word
```

```
batch_size = 32
data_loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

sample_x, sample_y = next(iter(data_loader))
print(f"Batch X tensor shape: {sample_x.shape}")
print(f"Batch Y tensor shape: {sample_y.shape}")
print(f"Sample Input IDs: {sample_x[0].tolist()}")
print(f"Sample Target ID: {sample_y[0].item()} ({idx2word[sample_y[0].ite
```

```
Batch X tensor shape: torch.Size([32, 5])
Batch Y tensor shape: torch.Size([32])
Sample Input IDs: [292, 354, 482, 311, 489]
Sample Target ID: 231 (in)
```

8. Transformer Language Model Architecture

In [10]:

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]

class TransformerLanguageModel(nn.Module):
    def __init__(self, vocab_size, d_model=128, nhead=4, num_layers=2, di
        super().__init__()
        self.d_model = d_model
        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=0)
        self.pos_encoder = PositionalEncoding(d_model)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=dim_feedforward,
            dropout=dropout,
            batch_first=True
        )
        self.transformer_encoder = nn.TransformerEncoder(encoder_layer, n
        self.output_head = nn.Linear(d_model, vocab_size)

    def generate_square_subsequent_mask(self, sz, device):
        mask = torch.triu(torch.ones(sz, sz, device=device) * float('-inf
        return mask

    def forward(self, x):
        seq_len = x.size(1)
        mask = self.generate_square_subsequent_mask(seq_len, x.device)
        embeddings = self.embedding(x) * math.sqrt(self.d_model)
        encoded = self.pos_encoder(embeddings)
        hidden_states = self.transformer_encoder(encoded, mask=mask)
        logits = self.output_head(hidden_states)
        return logits

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
model = TransformerLanguageModel(
    vocab_size=len(vocab),
    d_model=128,
    nhead=4,
    num_layers=2,
    dim_feedforward=256,
    dropout=0.1
).to(device)

print(f"Device: {device}")
print(f"Total model parameters: {sum(p.numel() for p in model.parameters(
print(model)
```

```
Device: cuda
Total model parameters: 401170
TransformerLanguageModel(
  (embedding): Embedding(530, 128, padding_idx=0)
  (pos_encoder): PositionalEncoding()
  (transformer_encoder): TransformerEncoder(
    (layers): ModuleList(
      (0-1): 2 x TransformerEncoderLayer(
        (self_attn): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=128,
out_features=128, bias=True)
        )
        (linear1): Linear(in_features=128, out_features=256, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
        (linear2): Linear(in_features=256, out_features=128, bias=True)
        (norm1): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
        (norm2): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
        (dropout1): Dropout(p=0.1, inplace=False)
        (dropout2): Dropout(p=0.1, inplace=False)
      )
    )
  )
  (output_head): Linear(in_features=128, out_features=530, bias=True)
)
```

9. Training the Transformer Model

In [11]:

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

epochs = 35
loss_log = []

for epoch in range(1, epochs + 1):
    model.train()
    epoch_loss = 0.0
    for x_batch, y_batch in data_loader:
        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)
        optimizer.zero_grad()
        logits = model(x_batch)
        last_position_logits = logits[:, -1, :]
        loss = criterion(last_position_logits, y_batch)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
```

```
        epoch_loss += loss.item()
    avg_epoch_loss = epoch_loss / len(data_loader)
    loss_log.append(avg_epoch_loss)
    if epoch % 5 == 0 or epoch == 1:
        print(f"Epoch [{epoch:02d}/{epochs}] - Loss: {avg_epoch_loss:.4f}
```

```
Epoch [01/35] - Loss: 6.4118
Epoch [05/35] - Loss: 3.1451
Epoch [10/35] - Loss: 0.7233
Epoch [15/35] - Loss: 0.1354
Epoch [20/35] - Loss: 0.0298
Epoch [25/35] - Loss: 0.0130
Epoch [30/35] - Loss: 0.0088
Epoch [35/35] - Loss: 0.0059
```

10. Text Generation Implementation

In [12]:

```
def generate_text(model, seed_text, max_words=20, sequence_length=5, temp
    model.eval()
    tokens = clean_and_tokenize(seed_text)
    if len(tokens) == 0:
        return "", "", ""

    generated_tokens = list(tokens)
    unk_id = word2idx['<unk>']
    pad_id = word2idx['<pad>']

    with torch.no_grad():
        for _ in range(max_words):
            current_window = generated_tokens[-sequence_length:]
            input_ids = [word2idx.get(w, unk_id) for w in current_window]
            if len(input_ids) < sequence_length:
                input_ids = [pad_id] * (sequence_length - len(input_ids))

            input_tensor = torch.tensor([input_ids], dtype=torch.long, de
            logits = model(input_tensor)
            last_logits = logits[0, -1, :] / max(temperature, 1e-5)
            probs = torch.softmax(last_logits, dim=-1)
            predicted_idx = torch.argmax(probs).item()
            predicted_word = idx2word.get(predicted_idx, '<unk>')
            generated_tokens.append(predicted_word)

    seed_str = " ".join(tokens)
    continuation_str = " ".join(generated_tokens[len(tokens):])
    full_str = " ".join(generated_tokens)
    return seed_str, continuation_str, full_str
```

11. Text Generation with Multiple Seed Sentences

In [13]:

```
seed_prompts = [
    "artificial intelligence and machine learning models",
    "astronauts aboard the international space station",
    "regular physical exercise balanced nutrition and"
]

results = []
for index, seed in enumerate(seed_prompts, start=1):
```

```
    seed_text, generated_text, full_output = generate_text(
        model=model,
        seed_text=seed,
        max_words=20,
        sequence_length=chosen_seq_len
    )
    results.append({
        'Index': index,
        'Seed Text': seed_text,
        'Generated Continuation': generated_text,
        'Full Generated Text': full_output,
        'Words Generated': len(generated_text.split())
    })
    print(f"==================== SEED EXPERIMENT {index} ================
    print(f"Seed Input:       {seed_text}")
    print(f"Generated Words:  {generated_text}")
    print(f"Combined Output:  {full_output}")
    print(f"Words Count:      {len(generated_text.split())}")
    print()
```

```
==================== SEED EXPERIMENT 1 ====================
Seed Input:       artificial intelligence and machine learning models
Generated Words:  are revolutionizing modern computer science and software
development deep learning neural networks require massive computing power and
graphics processing units
Combined Output:  artificial intelligence and machine learning models are
revolutionizing modern computer science and software development deep learnin
neural networks require massive computing power and graphics processing units
Words Count:      20

==================== SEED EXPERIMENT 2 ====================
Seed Input:       astronauts aboard the international space station
Generated Words:  conduct scientific experiments in zero gravity orbit to adv
human space exploration black holes possess immense gravitational pull such t
Combined Output:  astronauts aboard the international space station conduct
scientific experiments in zero gravity orbit to advance human space explorati
black holes possess immense gravitational pull such that
Words Count:      20

==================== SEED EXPERIMENT 3 ====================
Seed Input:       regular physical exercise balanced nutrition and
Generated Words:  sufficient sleep promote long term cardiovascular health an
mental vitality medical researchers develop novel vaccines antibiotics and
therapeutic treatments to
Combined Output:  regular physical exercise balanced nutrition and sufficient
promote long term cardiovascular health and mental vitality medical researche
develop novel vaccines antibiotics and therapeutic treatments to
Words Count:      20
```

12. Comparison of Generated Outputs for Different See Texts

Seed Prompt	Seed Domain	Continuation Characteristics	Domain Relevance	Coherence Obs
Seed 1: *"artificial intelligence and machine learning models"*	Technology / AI	Predicts words related to computing, modern computer science, and software systems.	High	Strong syntactic reflecting techno training corpora

Seed Prompt	Seed Domain	Continuation Characteristics	Domain Relevance	Coherence Obs
Seed 2: *"astronauts aboard the international space station"*	Space Exploration	Produces terms associated with zero gravity, scientific experiments, and space exploration.	High	Contextually coh and aligns with planetary/orbital sequences.
Seed 3: *"regular physical exercise balanced nutrition and"*	Health / Medicine	Connects with cardiovascular health, sleep, vitality, and medical wellness.	High	Matches health guidance structu learned from the document cluste

15. Advantages and Limitations of Transformer-Based T Generation

Key Advantage:

- **Global Receptive Field and Direct Dependency Modeling**: Unlike recurrent neural netwo where information degrades as it propagates sequentially through hidden states, self-attenti allows any word to attend directly to every other preceding word in constant path length $O(prevents information bottlenecking, captures long-range grammatical agreements, and enab massive parallelization across GPUs during training.

Key Limitation:

- **Quadratic Time and Memory Complexity with Sequence Length**: The self-attention calcu requires computing and storing an attention matrix of size $L \times L$, leading to $O(L^2)$ computational complexity and memory footprint. For very long documents or high-throughpu time generation, this quadratic scaling imposes substantial GPU memory constraints and lat costs compared to linear recurrent architectures.